Full-Agent Technical Write-up

Track: Full-Agent · **Kernel:** DeepSeek-V3 MoE forward · **Hardware:** NVIDIA B200 (sm_100) · **Language:** Triton

Team: GEMM People

Author: Jierui Xu

1. Solution Overview

We build a Triton implementation of the DeepSeek-V3 MoE forward path (no-aux routing → expert dispatch → grouped FP8 GEMM1 + SwiGLU → grouped FP8 GEMM2 + weighted scatter → bf16 output) entirely through a self-driving agent. The agent runs a three-phase pipeline — *Learn* → *Implement* → *Optimize* — under a strict harness that gates writes per phase, enforces commit-message conventions, and validates every artifact against JSON schemas.

2. Kernel Design & Optimizations

2.1 Kernel inventory

The production file `solution/triton/kernel.py` exposes five `@triton.jit` kernels orchestrated by one host launcher `run()`:

Kernel	Role	Output
`_routing_kernel`	Sigmoid + bias, group top-2 sum, group top-4 mask, masked global top-8, normalize, emit `-1`-sentinel local-expert ids	`topk_idx[T, 8]`, `topk_w[T, 8]`
`_dispatch_count_kernel`	Per-token atomic_add into `hist[E_LOCAL]`	expert histogram
`_dispatch_scatter_kernel`	Per-token atomic counter slot → write `(t_sorted, gamma_sorted)`	sorted dispatch arrays
`_gemm1_swiglu_kernel`	FP8×FP8 block-scaled GEMM1 fused with `silu(gate) · up`, bf16 intermediate store	`[sum_Tk, I=2048]` bf16
`_gemm2_scatter_kernel`	bf16×FP8 block-scaled GEMM2 with γ-weighted `atomic_add` into fp32 accumulator	`[T, H]` fp32
`_cast_fp32_to_bf16_kernel`	Final fp32→bf16 cast directly into the caller's output	`[T, H]` bf16

2.2 Memory and numerics

- **Block-scaled FP8.** Both GEMMs load FP8 operands, dequantize per-128 block scales in registers, cast to bf16, and run `tl.dot(..., out_dtype=fp32)`. Pre-scaling operands inside the K-loop is essential — we observed that hoisting the scale to the post-dot epilogue defeats Triton's dequant↔dot pipeline fusion (long-class regressed +140 %).
- **bf16 intermediate buffer.** GEMM1 stores its SwiGLU result as bf16 instead of fp32; GEMM2 reads bf16 LHS. Halves GEMM2 LHS HBM traffic (~3.2 GB→1.6 GB at seq=14107) without measurable accuracy loss.
- **fp32 atomic accumulator.** GEMM2 `tl.atomic_add`s into fp32 then a separate cast kernel emits bf16. We tried bf16 atomics directly into the user output — B200 falls back to a CAS loop and the long class regressed +38 %.
- **Per-seq-len frozen tile config.** A 1-line dict lookup (`_GEMM1_BEST` / `_GEMM2_BEST`) selects `(BLOCK_M, BLOCK_N, num_stages, num_warps)` per workload, populated once by an autotune sweep and never re-swept.

2.3 Routing & dispatch in Triton

Routing and dispatch — typically a chain of ~15+ small torch CUDA kernels (`gatherTopK×3`, `scatter`, `bincount`, `argsort`, `cumsum`, `index_select×3`, `masked_fill`, …) — each pay ~10 µs of launch overhead and dominate at short sequences. The agent replaced this entire chain with three Triton kernels:

1. **One-token-per-program routing.** Each program loads the 256-wide logit row, computes group top-2 in a single 8×32 reshape, iteratively `argmax`es to pick top-4 groups and global top-8 experts, and writes a `-1` sentinel for non-local experts so host dispatch needs only `e ≥ 0`.
2. **Two-pass atomic dispatch.** Pass 1 builds the per-expert histogram via `atomic_add(hist[e])`; the host runs a 32-element `cumsum` (negligible). Pass 2 obtains a slot via `atomic_add(counter[e])` and writes `(t_sorted[slot], gamma_sorted[slot])`.

3. Performance Analysis

3.1 Workload classification

Phase-3.A profiled all 19 workloads via `modal_profile` (end-to-end) and `modal_ncu` (per-kernel deep-dive) and produced three classes:

Class	seq_len range	Dominant bottleneck	Rep
`short_latency_bound`	1–80 (16 wls)	Routing + dispatch (≈44 % of 0.58 ms) — launch overhead ≫ math	1
`medium_mixed`	901 (1 wl)	Balanced GEMM1 (43.6 %) + GEMM2 (37.9 %)	901
`long_compute_bound`	11948–14107 (2 wls)	GEMM2 (55.6 %) — GEMM2 LHS bytes dominate	14107

3.3 Optimization trajectory (15 feature iterations + 1 integration)

Landed wins (in order of impact):

iter	technique	target class	mean Δ
10	Replace torch dispatch chain with 2 Triton kernels	short	−17.5 %
1	Fuse DSv3 routing into a single Triton kernel	short	−14 %
7	Drop `is_local.any()` CPU↔GPU sync	short	−4.3 %
12	Fused fp32→bf16 cast kernel	short	−3.5 %
9	Emit `-1`-sentinel local-expert id from routing	short	−3 %
3	bf16 GEMM1→GEMM 2 intermediate buffer	long	−2 %

4. Tools & Languages

- **Triton** — chosen because the contest restricts dependencies and the entry point is `run()` in a Python module. Triton's pipeline scheduler, block-scaled `tl.dot`, and `atomic_add` cover everything needed; CUTLASS / CuTe-DSL would have given more peak FLOPs but at the cost of a much larger search space for the agent.

5. Agent Architecture & Workflow

5.1 Three-phase harness

```
context/  —▶  Phase 1 (Learn)     —▶ knowledge/      (structured notes, citations
required)
input/    —▶  Phase 2 (Implement) —▶ solution/       (5 stages: fuse → spec → kernel
→ verify → autotune freeze)
solution/ —▶  Phase 3 (Optimize)  —▶ solution/ + ITERATIONS.md + iterations.jsonl
```

Each phase is entered through a slash-command prompt (`.claude/commands/phaseN-*.md`). A `pre_edit` hook reads `.claude/state/current_phase` and **rejects (exit 2) any write to a directory belonging to another phase** — Phase 2 cannot modify `knowledge/`; Phase 3 cannot touch `bench/modal_autotune.py` unless `autotune_change_reason.txt` exists ("freeze rule"). A `pre_bash` hook rejects unsafe `git`, `rm -rf`, `pip install`, and `modal run` invocations missing the conda-env prefix or the `-m` module flag.

5.2 Phase 1 — knowledge curation

The agent is forbidden to write code or run benchmarks. It must produce one Markdown file per topic in `knowledge/README.md` (tiling, memory hierarchy, async pipelines, reductions, numerics, quantization, autotune-space, MoE algorithm variants, MoE data layouts, MoE routing) using a frozen frontmatter template (`topic / applies_to / source / confidence`) plus required body sections (Summary / When to use / Code pattern / Tradeoffs / Pitfalls). Every claim must cite a path under `context/` — uncited claims fail the phase. The output is consumed by Phase 2 (kernel impl) and Phase 3's stall-recovery rule.

5.3 Phase 2 — implementation in five gated stages

1. **2.1 Fusion judgment** — rewrite the reference PyTorch with each fused subgraph wrapped in a named function, validated against the original on a fixed seed.
2. **2.2 Subgraph specs** — emit `subgraph_specs.json` (Draft-07 schema-validated) with op lists, shapes, dtypes, and verbatim source snippets; deduplicate by signature.
3. **2.3 Kernel impl** — one Triton kernel per spec; *naive is good*. Cross-subgraph fusion is forbidden here (a Phase-3 decision).
4. **2.4 Verification** — produce `rules_check.json`, run `tools/check_rules.py` (exit 0), then `bash scripts/bench.sh baseline`. Commit `[baseline]`.
5. **2.5 Autotune freeze** — sweep all 19 workloads × 2 kernels × ~20 pruned configs via `modal_autotune.py`, copy winners into a per-seq dict at the top of `kernel.py`, re-bench, commit `[baseline-autotuned]`. Phase 3 may not re-sweep without an algorithmic change.

5.4 Phase 3 — iteration loop

Three sub-phases: **3.A classify** (profile-driven, no code edits), **3.B per-class iterate**, **3.C integrate**.

The 3.B iteration protocol enforced by the harness:

1. Pick one class and one bottleneck from its profile; write `class_id` + target into `ITERATIONS.md` *before* editing.
2. Apply **one** technique per iteration; cite the knowledge file it came from.
3. Bench 5× on the representative; record the median.
4. Regression-guard against every previously-optimized class (default 3 % tolerance).
5. Re-profile (`modal_profile` + `modal_ncu`) to find the new dominant stage.
6. Append a row to **both** `iterations.jsonl` (schema-validated) and `ITERATIONS.md`; `tools/validate_artifacts.py` keeps them in sync.
7. Commit with prefix `[iter N][<class_id>] …`.
8. **Cleanup gate**: every fifth iteration triggers an `[iter-N cleanup]` commit (dead-code drop, stale-config drop, debug-print removal, `ITERATIONS.md` collapse). The next feature iteration is blocked until cleanup lands.